

====Git教程====

分布式版本控制工具 vs 集中式版本控制工具

Git常见命令

Git分支，分支特性，分支创建，分支转换，分支合并，代码合并冲突解决

IDE集成Git

====Github====

创建远程库

代码推送 PUSH

代码拉取 PULL

代码克隆 Clone

SSH免密登录

IDE集成Github

====Gitee码云====

码云创建远程库

IDE集成码元

码云连接Github进行代码的复制，迁移

====Gitlab====

Gitlab服务器搭建和部署

IDE集成Github

1、Git简介

1.1 概述

开源、分布式版本控制系统



Git is a **free and open source** distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is **easy to learn** and has a **tiny footprint with lightning fast performance**. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like **cheap local branching**, convenient **staging areas**, and **multiple workflows**.

1.2 版本控制

版本控制：记录文件的内容变化，以便将来查阅特定版本修订情况的系统。

记录修改历史记录，让用户能够查看历史版本，方便版本切换。

名称	修改日期
 XXX毕业论文.docx	2021/4/7 15:01
 XXX毕业论文改1.docx	2021/4/7 15:01
 XXX毕业论文改2.docx	2021/4/7 15:01
 XXX毕业论文完成版1.docx	2021/4/7 15:01
 XXX毕业论文完成版2.docx	2021/4/7 15:01
 XXX毕业论文最终版1.docx	2021/4/7 15:01
 XXX毕业论文最终版2.docx	2021/4/7 15:01
 XXX毕业论文最最终版1.docx	2021/4/7 15:01
 XXX毕业论文最最最终版1.docx	2021/4/7 15:01
 XXX毕业论文最最最终绝对不修改版1.docx	2021/4/7 15:01
 XXX毕业论文最最最终绝对不修改版2.docx	2021/4/7 15:01
 XXX毕业论文最最最终绝对不修改版修改就辍学版.docx	2021/4/7 15:01
 XXX毕业论文最最最终绝对不修改版修改就辍学版2.docx	2021/4/7 15:01
 给老师的一封信-辍学是怎么炼成的.docx	2021/4/7 15:01
 遗书.docx	2021/4/7 15:01

从个人开发过渡到团队开发。

1.3 版本控制工具

*****本地版本控制*****

就是存储在本地的若干版本。即上图。

*****集中式版本控制工具*****

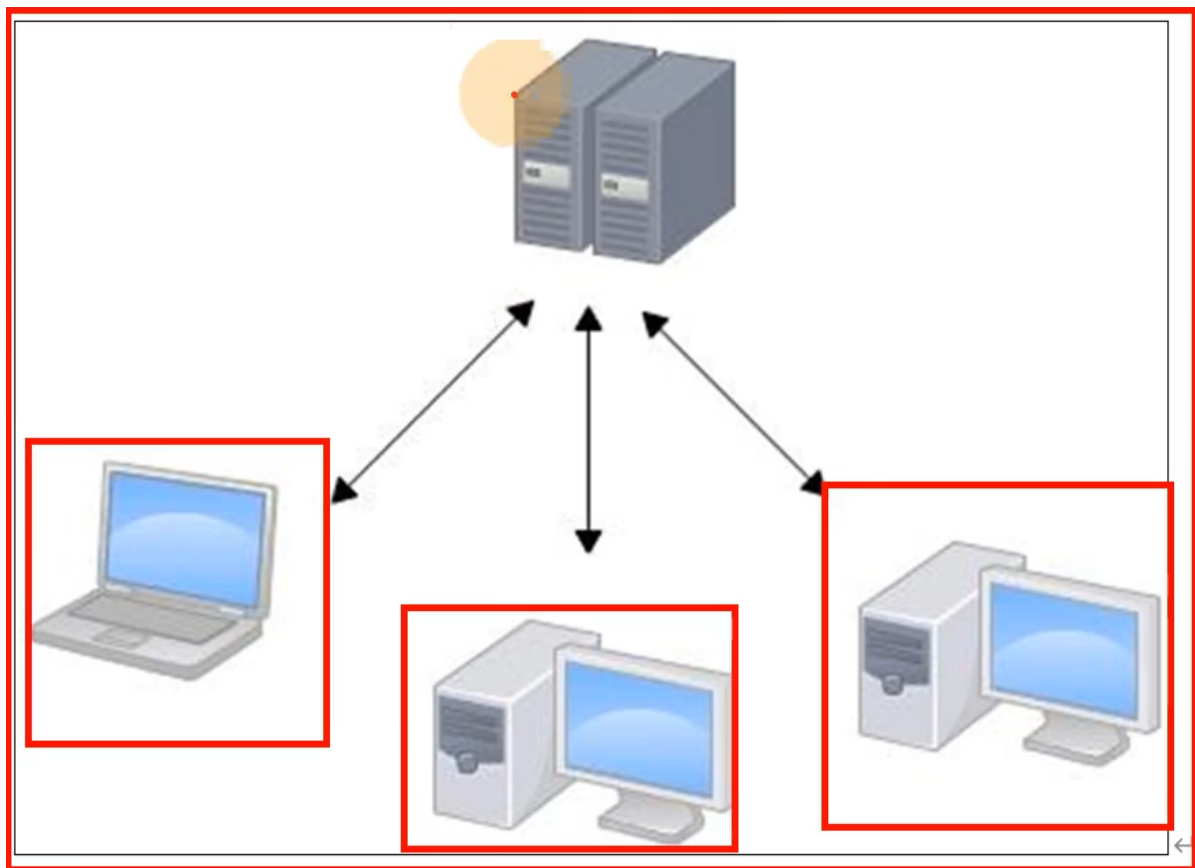
所有的版本都存在于服务器，本地只有自己的历史版本。不联网，无法获取到全部版本。

CVS、SVN(Subversion)、VSS.....

集中化的版本控制系统诸如 CVS、SVN 等，都有一个单一的集中管理的服务器，保存所有文件的修订版本，而协同工作的人们都通过客户端连到这台服务器，取出最新的文件或者提交更新。多年以来，这已成为版本控制系统的标准做法。

这种做法带来了许多好处，每个人都可以在一定程度上看到项目中的其他人正在做些什么。而管理员也可以轻松掌控每个开发者的权限，并且管理一个集中化的版本控制系统，要远比在各个客户端上维护本地数据库来得轻松容易。

事分两面，有好有坏。这么做显而易见的缺点是中央服务器的单点故障。如果服务器宕机一小时，那么在这一小时内，谁都无法提交更新，也就无法协同工作。



*****分布式版本控制工具*****

每个人都拥有全部的代码。所有版本信息仓库同步到本地的每个用户，本地查看所有版本历史，可以离线在本地提交，联网时Push到相应的服务器或用户哪里。

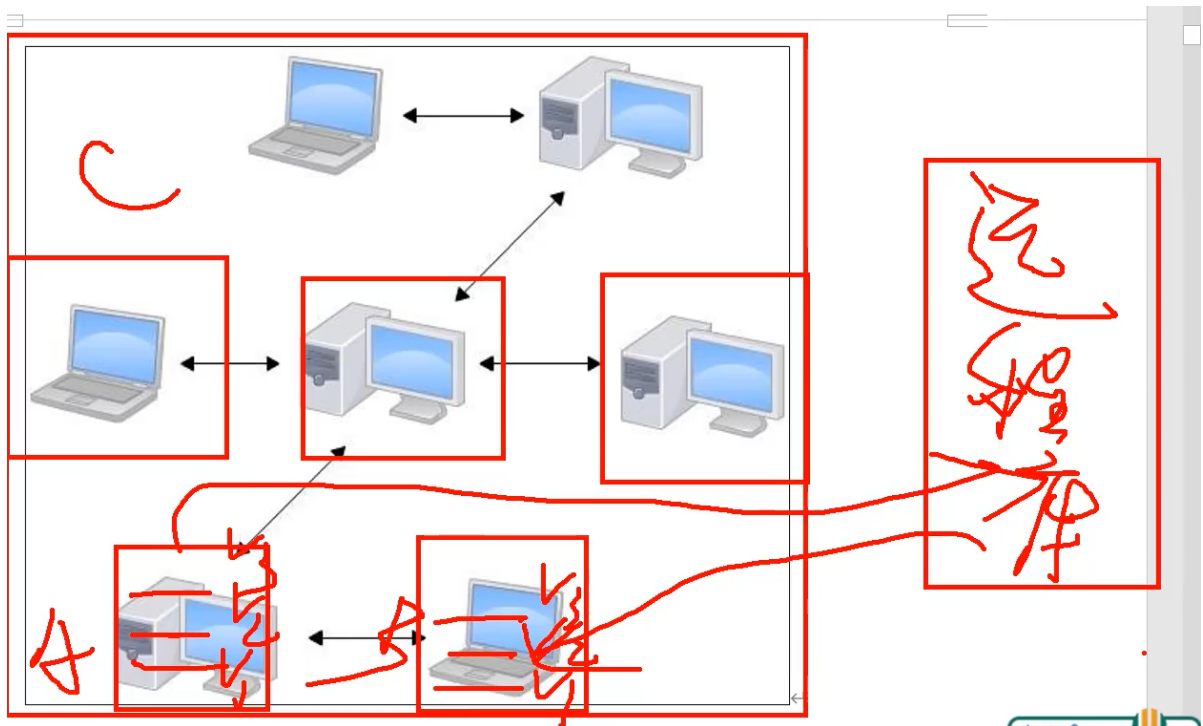
➤ 分布式版本控制工具

Git、Mercurial、Bazaar、Darcs.....

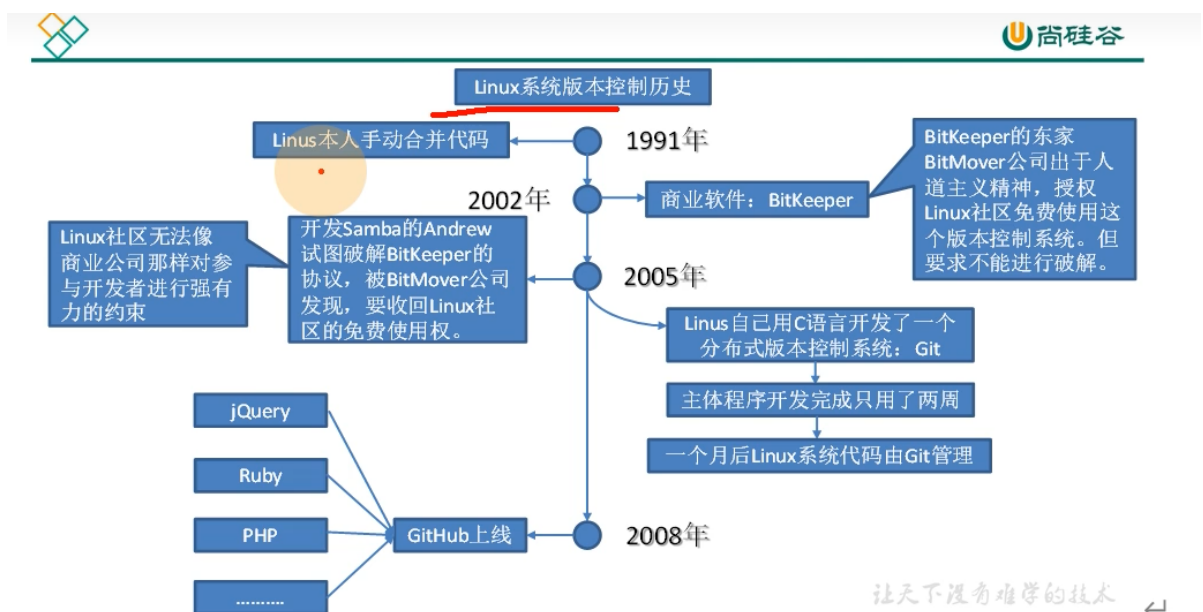
像 Git 这种分布式版本控制工具，客户端提取的不是最新版本的文件快照，而是把代码仓库完整地镜像下来（本地库）。这样任何一处协同工作用的文件发生故障，事后都可以用其他客户端的本地仓库进行恢复。因为每个客户端的每一次文件提取操作，实际上都是一次对整个文件仓库的完整备份。

分布式的版本控制系统出现之后，解决了集中式版本控制系统的缺陷：

1. 服务器断网的情况下也可以进行开发（因为版本控制是在本地进行的）
2. 每个客户端保存的也都是整个完整的项目（包含历史记录，更加安全）



1.4 历史发展



1.5 工作区和代码托管中心



工作区：代码存放的位置。删除了就真的删除了

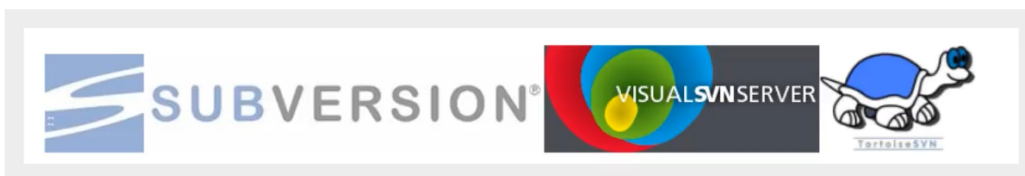
暂存区：Git暂存的代码。删除了，还没提交，删除了还存在工作区。

本地库：无法再次直接删除。在之前版本中，还存在记录。可以直接删除本地库，跑路。hhh

远程库：把本地库的内容推送到远程库。

Git与SVN最主要区别

SVN是集中式版本控制系统，版本库是集中放在中央服务器的，而工作的时候，用的都是自己的电脑，所以首先要从中央服务器得到最新的版本，然后工作，完成工作后，需要把自己做完的活推送到中央服务器。集中式版本控制系统是必须联网才能工作，对网络带宽要求较高。



Git是分布式版本控制系统，没有中央服务器，每个人的电脑就是一个完整的版本库，工作的时候不需要联网了，因为版本都在自己电脑上。协同的方法是这样的：比如说自己在电脑上改了文件A，其他人也在电脑上改了文件A，这时，你们两之间只需把各自的修改推送给对方，就可以互相看到对方的修改了。Git可以直接看到更新了哪些代码和文件！

Git是目前世界上最先进的分布式版本控制系统。

##

1.6 代码托管中心

代码托管中心是基于网络服务器的远程代码仓库，一般我们简单称为**远程库**。↵

➤ 局域网↵

✓ GitLab↵

➤ 互联网↵

✓ GitHub (外网)↵

✓ Gitee 码云 (国内网站)↵

1.7 安装与配置

[Git安装全教程 \(Win11\) - 知乎 \(zhihu.com\)](#)

Git bash : unix与linux风格命令行

Git cmd: windows风格

Git GUI: 图形界面

环境变量只是为了全局使用而已。

<https://www.liaoxuefeng.com/wiki/896043488029600> git 简单学习路径

2 Git常用命令

2.1 Linux常见命令

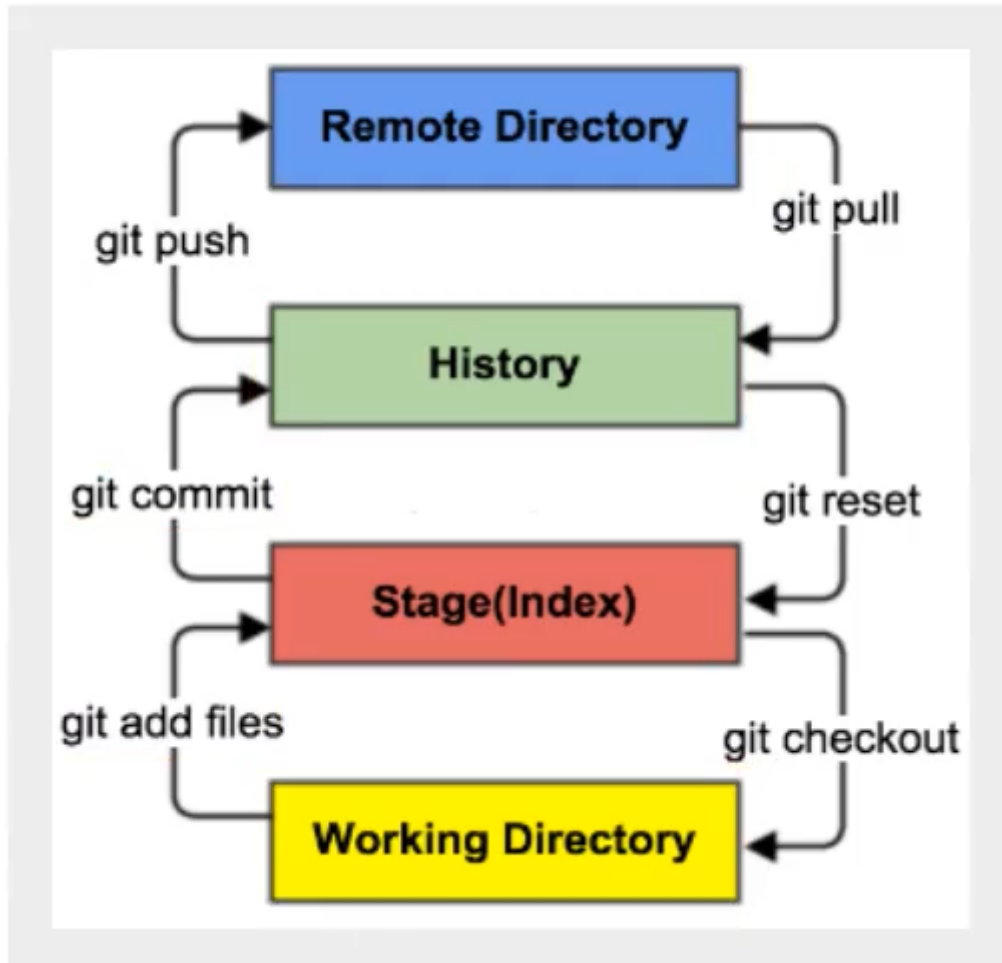
- **cd**: 改变目录
- **cd..** : 回到上一个目录, 直接cd进入默认目录
- **pwd** : 当前文件所在路径
- **ls** : 查看当前文件夹下的文件
- **ls(l 两个l)** : 列出当前目录下的所有文件, 只不过列出的内容更为详细
- **touch** : 新建一个文件 eg: touch index.js
- **rm** : 删除文件 eg: rm index.js
- **mkdir** : 新建一个文件夹 mkdir <文件夹名>
- **rm -r** : 删除一个文件夹 rm -r <文件夹>
- 别用 **rm -rf /** / 清除根目录下所有文件
- **mv** : 移动文件, mv <移动的文件> <新文件夹>
- **reset** : 初始化终端
- **clear** : 清屏 (Windows: cls)
- **history** : 查看历史命令
- **help**
- **exit**

- 符号 # 表示注释
- 查看隐藏文件 || -a (||只能查看未隐藏文件)

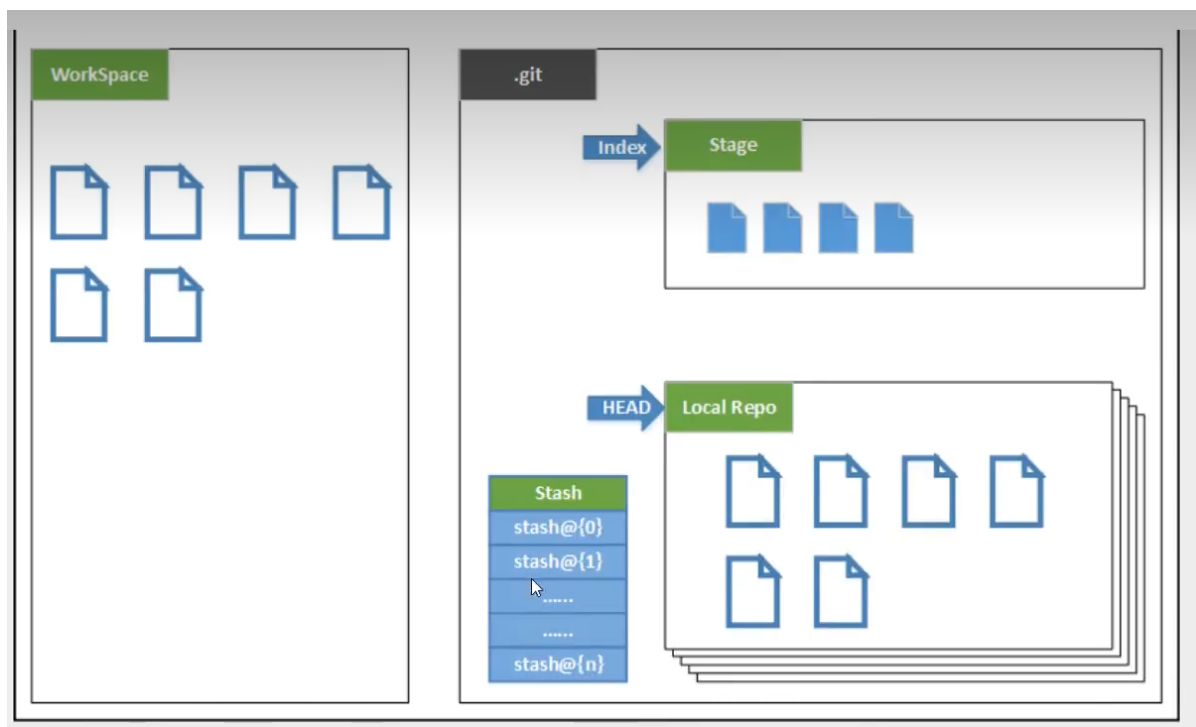
重要提示： 当文件夹中含有空格时候，要用单引号引起来该文件夹路径 eg: `cd 'code files'`

2.2 Git原理

Git分为三个工作区：工作目录 (working directory)，暂存区(Stage/Index)，资源库 (Repository或者Git Directory)，若再加上远程的Git长裤可以分为四个区域。



- Workspace：工作区，就是你平时存放项目代码的地方
- Index / Stage：暂存区，用于临时存放你的改动，事实上它只是一个文件，保存即将提交到文件列表信息
- Repository：仓库区（或本地仓库），就是安全存放数据的位置，这里面有你提交到所有版本的数据。其中HEAD指向最新放入仓库的版本
- Remote：远程仓库，托管代码的服务器，可以简单的认为是你项目组中的一台电脑用于远程数据交换



2.3 常用的Git命令

- `git config -l` 查看Git的一些配置文件
- `git config --system --list` 系统的一些配置
- `git config --global --list` 本地的配置

所有的配置文件其实都保存在本地。

查看不同级别的配置文件：

```
#查看系统config
git config --system --list

#查看当前用户（global）配置
git config --global --list
```

Git相关的配置文件：

1)、`Git\mingw64\etc\gitconfig`：Git 安装目录下的 `gitconfig --system` 系统级

2)、`C:\Users\Administrator\.gitconfig` 只适用于当前登录用户的配置 `--global` 全局

这里可以直接编辑配置文件，通过命令设置后会响应到这里。|

必要的配置： 仅在使用前配置一次

本地配置：`git config --global --list`

系统配置：`git config --system --list`

可以去相应的文件夹下查看 `.gitconfig` 里面的内容

```
git config --global user.name <用户名>
```

```
git config --global user.email <邮箱>
```

签名的作用是为了区分不同作者的身份，首次安装必须配置用户前面，否则无法提交代码。（与以后登录Github或其他代码托管中心的账号无关）

1) 基本语法

git init 初始化本地库

由于在安装软件时配置了能在文件夹打开git,所以只需要在相应文件夹下打开git即可直接到相应的目录。

```
sung@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files
$ git init
Initialized empty Git repository in D:/Code Files/Git Files/.git/

sung@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ |
```

ll -a 查看隐藏文件,

cd <文件名>

2) 克隆文件

git clone

3) 文件操作

GIT文件操作

文件4种状态

版本控制就是对文件的版本控制，要对文件进行修改、提交等操作，首先要知道文件当前在什么状态，不然可能会提交了现在还不提交的文件，或者要提交的文件没提交上。

- **Untracked:** 未跟踪, 此文件在文件夹中, 但没有加入到git库, 不参与版本控制. 通过 **git add** 状态变为 **Staged**.
- **Unmodify:** 文件已经入库, 未修改, 即版本库中的文件快照内容与文件夹中完全一致. 这种类型的文件有两种去处, 如果它被修改, 而变为 **Modified**, 如果使用 **git rm** 移出版本库, 则成为 **Untracked** 文件
- **Modified:** 文件已修改, 仅仅是修改, 并没有进行其他的操作. 这个文件也有两个去处, 通过 **git add** 可进入暂存 **staged** 状态, 使用 **git checkout** 则丢弃修改过, 返回到 **unmodify** 状态, 这个 **git checkout** 即从库中取出文件, 覆盖当前修改!
- **Staged:** 暂存状态. 执行 **git commit** 则将修改同步到库中, 这时库中的文件和本地文件又变为一致, 文件为 **Unmodify** 状态. 执行 **git reset HEAD filename** 取消暂存, 文件状态为 **Modified**

查看文件状态

```
git status (git status <filename>)
```

添加所有文件到暂存区:

```
git add . (git add <filename>)
```

提交一些信息到本地仓库:

```
git commit -m "日志信息" <filename> (-m 表示message)
```

```
eg: git commit -m "first commit" test1.py
```

提交后会产生一个版本号,

删除暂存区文件:

```
git rm --cached <filename>
```

查看版本日志信息:

```
git reflog
```

```
git log 查看详细信息, 能看到谁提交的库
```

vim命令:

修改为文件

```
vim test1.py
```

进入编辑器后按 字母“i”即可进入编辑状态（此时左下角会出现“插入”）

退出的时候分为4种情况：保存退出、正常退出、不保存退出以及强制退出

2.1: 保存退出: 按“Esc”键后 此时的“插入”会消失, 然后按Shift+zz 就可以保存修改内容并退出

2.2: 不保存退出: 当修改修改了一部分内容后发现修改错了, 此时就会进行不保存退出 按“Esc”键后, 再输入“:”之后在输入命令时直接输入“q!”

2.3: 强制退出: 按“Esc”键后, 再输入“:”之后在输入命令时直接输入“!”

2.4: 正常退出: 按“Esc”键后, 再输入“:”之后在输入命令时直接输入“q”

修改后, 如果对以及编辑的一行进行修改, 只能显示一行新增, 一行删除。

查看相应的文件内容 `cat <filename>`

`tail -n 1 filename.txt` 查看最后一行

```
sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git add test1.py

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   test1.py

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git commit -m "first commit" test1.py
[master (root-commit) e079a81] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 test1.py

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git status
On branch master
nothing to commit, working tree clean

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$
```

```
sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git log
commit 5fa1ca2d8c9c3628480fc0597d8770721d0a4116 (HEAD -> master)
Author: 少主 <sungt2021@gamil.com>
Date:   Thu Aug 24 19:11:53 2023 +0800

    second commit

commit e079a817a9fdc58f62d58b3801ab9f0bbb049901
Author: 少主 <sungt2021@gamil.com>
Date:   Thu Aug 24 19:05:53 2023 +0800

    first commit

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git reflog
5fa1ca2 (HEAD -> master) HEAD@{0}: commit: second commit
e079a81 HEAD@{1}: commit (initial): first commit
```

4) 忽略文件配置

一些文件不需要提交, 例如配置文件等, 可以在主目录下建立 .gitignore文件, 其配置规则如下:

1. .gitignore文件中的空行或以#开始的行会被忽略
2. 可以使用linux通配符, 例如 (*) 代表任意多个字符, (?) 代表一个字符, [] 方括号代表可选字符范围, { } 代表可选字符串
3. 如果名称的前面有一个 (!) 代表例外规则, 即不被忽略
4. 如果名称的最前面有一个 (/) 表示要忽略的文件再此目录下, 但是子目录中的文件不被忽略

5. 如果名称的最后面有一个 (/), 表示要忽略的是此目录下该名称的子目录, 而非文件。(默认文件或目录都忽略)

```
# 为注释
*.txt      #忽略所有以.txt结尾的文件
!lab.txt   # lab.txt文件除外
/temp      #往上走, 仅仅忽略目录下的其他文件, 不包括temp文件
build/     #忽略build文件夹下的所有文件
doc/*.txt  #忽略doc/.txt结尾的文件, 但不会忽略doc/m/arch.txt文件
```

<https://github.com/github/gitignore>

常用的忽略文件链接

2.4 版本穿梭

通过 `git reflog` 查看历史的版本日志

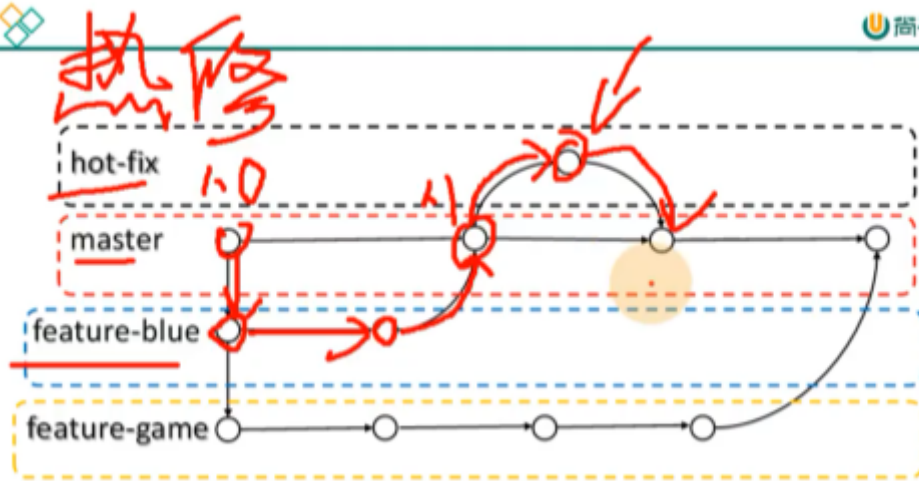
```
git reset --hard <版本号>
    可以看到指针指向了穿越回去的版本
    head/master/<filename>
    可以穿越到各个版本中
git log --pretty=oneline
    应该是用来说只显示log的第一行信息
```

2.5 分支

1) 概述

• 4.1 什么是分支

在版本控制过程中, 同时推进多个任务, 为每个任务, 我们就可以创建每个任务的单独分支。使用分支意味着程序员可以把自己的工作从开发主线上分离开来, 开发自己分支的时候, 不会影响主线分支的运行。对于初学者而言, 分支可以简单理解为副本, 一个分支就是一个单独的副本。(分支底层其实也是指针的引用)



好处是可以并行推进多个任务，且某一任务失败不会影响主任务

2) 分支操作

命令名	作用
git branch <分支名> git checkout -b <分支名> 此外还有， git switch -c <分支名>	创建分支/创建并切换分支
git branch -v	查看分支
git checkout <分支名> 更合理的 git switch <分支名>	切换分支;;创建并切换分支
git merge <分支名>	将指定的分支的内容融合到当前分支
git branch -d/-D <分支名>	删除指定分支/D表示未被合并的分支

eg:

```
git branch -v
git branch hello
git checkout master
git merge hello
```

```

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git branch main

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git branch -v
  main    5falca2 second commit
* master  5falca2 second commit

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git checkout main
Switched to branch 'main'

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git status
On branch main
nothing to commit, working tree clean

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git reflog
5falca2 (HEAD -> main, master) HEAD@{0}: checkout: moving from master to main
5falca2 (HEAD -> main, master) HEAD@{1}: reset: moving to 5falca2
e079a81 HEAD@{2}: reset: moving to e079a81
5falca2 (HEAD -> main, master) HEAD@{3}: commit: second commit
e079a81 HEAD@{4}: commit (initial): first commit

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ vim test1.py

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ cat test1.py
name hello i221
hello name i name
this is add by main branch

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   test1.py

no changes added to commit (use "git add" and/or "git commit -a")

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git add test1.py

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git commit -m "main first commit" test1.py
[main 762c0e8] main first commit
 1 file changed, 1 insertion(+)

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git status
On branch main
nothing to commit, working tree clean

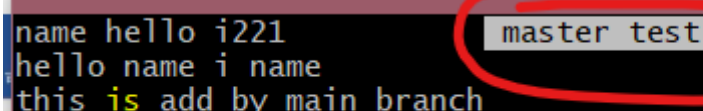
```

3) 分支合并冲突

合并分支时，两分支在**同一文件的同一位置**有两套完全不同的修改，必须人为确定使用哪一个分支。

具体实例如下：

对master下的test1.py文件进行修改：增加master test 代码



```

name hello i221
hello name i name
this is add by main branch
~
~

```

已提交至本地库:

```
sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git commit -m "master test" test1.py
[master 9e8c750] master test
1 file changed, 1 insertion(+), 1 deletion(-)

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ cat test1.py
name hello i221          master test
hello name i name
this is add by main branch

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git reflog
9e8c750 (HEAD -> master) HEAD@{0}: commit: master test
762c0e8 (main) HEAD@{1}: merge main: fast forward
5falca2 HEAD@{2}: checkout: moving from main to master
762c0e8 (main) HEAD@{3}: commit: main first commit
5falca2 HEAD@{4}: checkout: moving from master to main
5falca2 HEAD@{5}: reset: moving to 5falca2
e079a81 HEAD@{6}: reset: moving to e079a81
5falca2 HEAD@{7}: commit: second commit
e079a81 HEAD@{8}: commit (initial): first commit
```

对main分支进行修改:

```
name hello i221
hello name i name    main test
this is add by main branch
~
```

分支合并, 冲突, 且最后一行代码显示合并冲突

```

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ cat test1.py
name hello i221
hello name i name      main test
this is add by main branch

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   test1.py

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git commit -m "main test" test1.py
[main 69aab92] main test
 1 file changed, 1 insertion(+), 1 deletion(-)

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (main)
$ git checkout master
Switched to branch 'master'

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git merge main
Auto-merging test1.py
CONFLICT (content): Merge conflict in test1.py
Automatic merge failed; fix conflicts and then commit the result.

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   test1.py

no changes added to commit (use "git add" and/or "git commit -a")

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master|MERGING)
$ |

```

修改方法:

vim

下图中<<<<HEAD与===中间代码与当前分支的代码，===到<<<<<为下一分支的代码，将其中需要保留的部分留下，不需要的部分删除即可。

```

<<<<<<< HEAD
name hello i221          master test
hello name i name
=====
name hello i221
hello name i name      main test
>>>>>>> main
this is add by main branch
~

```

注意： 提交时候，不能加入文件名，

```

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master|MERGING)
$ vim test1.py

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master|MERGING)
$ git add test1.py

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master|MERGING)
$ git commit -m "merge test" test1.py
fatal: cannot do a partial commit during a merge.

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master|MERGING)
$ git commit -m "merge test"
[master adce7b2] merge test

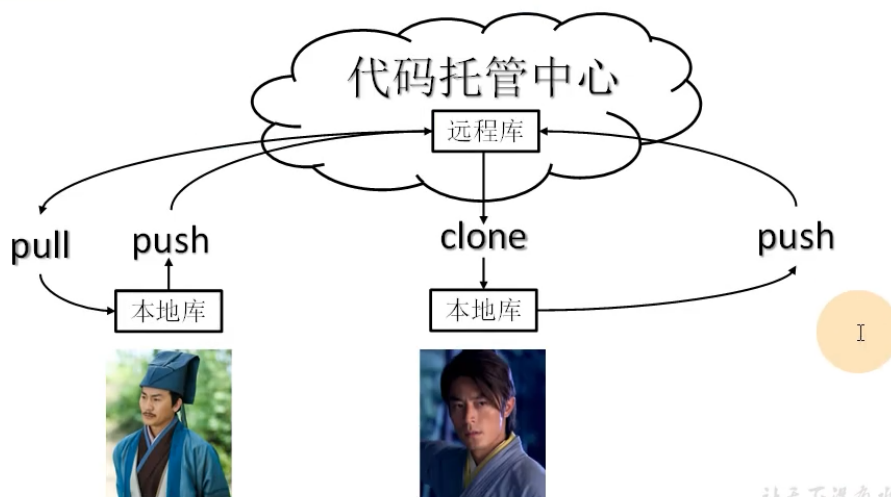
sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git reflog
adce7b2 (HEAD -> master) HEAD@{0}: commit (merge): merge test
9e8c750 HEAD@{1}: checkout: moving from main to master
69aab92 (main) HEAD@{2}: commit: main test
762c0e8 HEAD@{3}: checkout: moving from master to main
9e8c750 HEAD@{4}: commit: master test
762c0e8 HEAD@{5}: merge main: Fast-forward
5fa1ca2 HEAD@{6}: checkout: moving from main to master
762c0e8 HEAD@{7}: commit: main first commit
5fa1ca2 HEAD@{8}: checkout: moving from master to main
5fa1ca2 HEAD@{9}: reset: moving to 5fa1ca2
e079a81 HEAD@{10}: reset: moving to e079a81
5fa1ca2 HEAD@{11}: commit: second commit
e079a81 HEAD@{12}: commit (initial): first commit

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ |

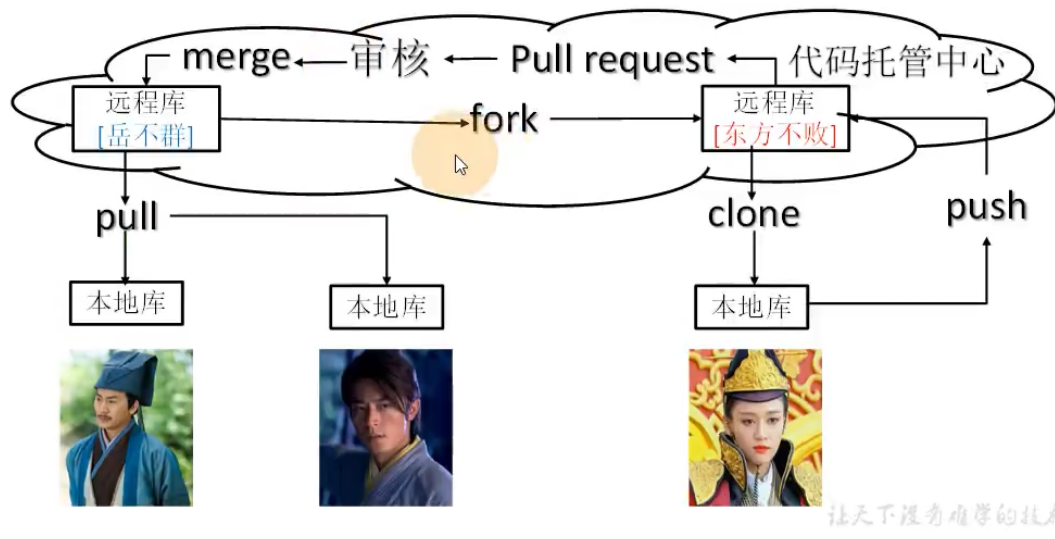
```

2.6 团队协作

• 5.1 团队内协作



5.2 跨团队协作



3 Git&Github

3.1 创建远程仓库

即创建GitHub仓库的常规操作

3.2 创建别名

觉得链接过长，可以创建链接别名。进行拉取和推送操作。

常见命令	说明
git remote -v	查看远程库别名
git remote add <别名>	创建远程库别名
git push <别名/url> <分支> 网络要求	将分支推送到远程库
git pull <别名/url> <远程库分-名> 更新本地库代码	将远程端的分支拉取回本地
git clone	克隆远程代码到本地

```
sung@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git remote -v

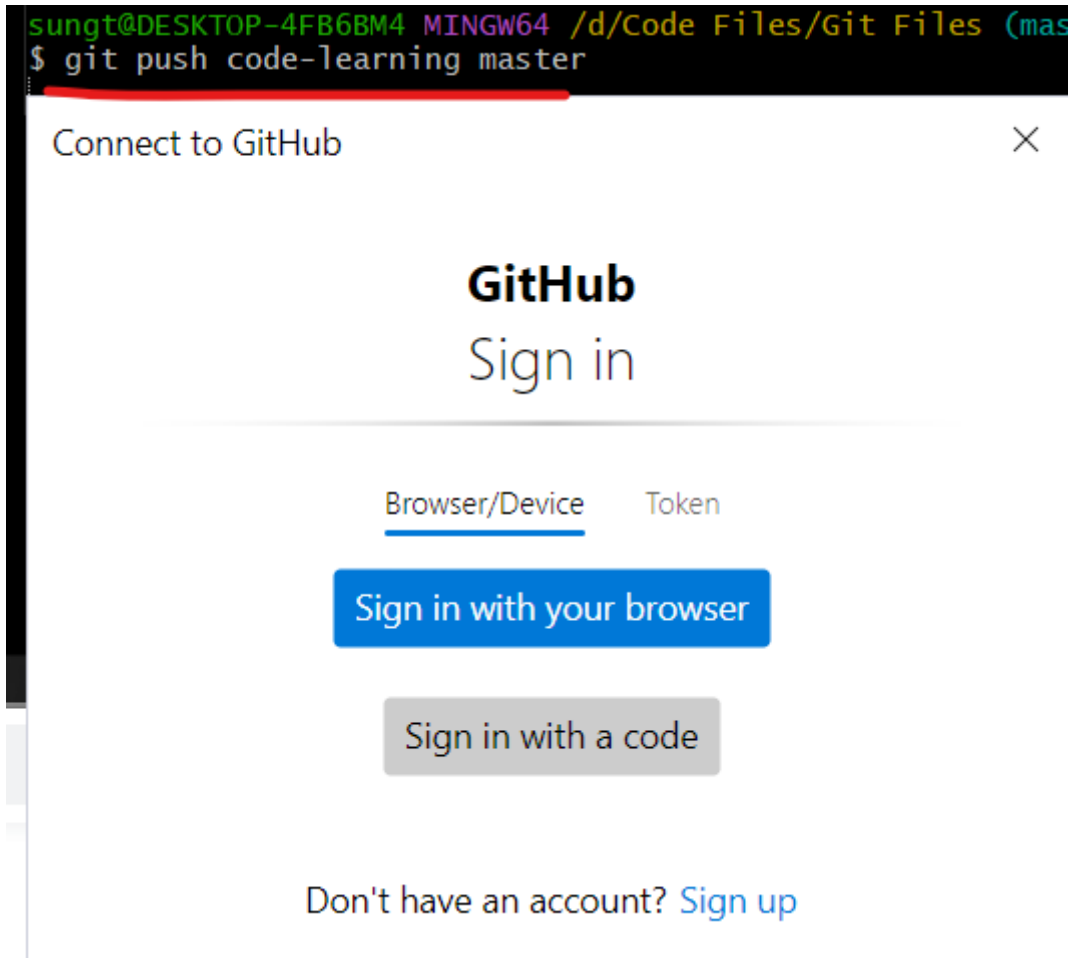
sung@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git remote add code-learning https://github.com/Future-001/Code-Learning.git

sung@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git remote -v
code-learning https://github.com/Future-001/Code-Learning.git (fetch)
code-learning https://github.com/Future-001/Code-Learning.git (push)
```

3.3 推送本地库

命令： `git push <别名/url> <分支名>`

`git push <别名> <本地分支名>:<远程新分支名>` 本来默认应该是远程的同名分支，然后现在可以进行修改



3.4 拉取远程代码

命令： `git pull <别名/url> <远程库分支名>` 将远程端的分支拉取回本地，更新本地代码

这里还有一种方法，从远程 `git pull` 拉取不成功，那么

```
git checkout -b dev origin/dev
```

```
== git switch -c <分支名> 远程名/远程分支
```

创建远程分支到本地。

如果同时多人修改了这个分支，那么会导致，合并冲突。那么先`git pull` 拉取最新，然后本地合并，在提交

```
git pull 远程分支名 本地库 --allow-unrelated-histories
```

`git pull`失败了，提示`no tracking information` 一般是没有指定本地`dev`分支与远程`origin/dev`分支的链接，根据提示，设置`dev`和`origin/dev`的链接：

```
git branch --set-upstream-to=远程库名/分支名 本地分支名
```

```
sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ git pull code-learning master
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 714 bytes | 44.00 KiB/s, done.
From https://github.com/Future-001/Code-Learning
* branch          master      -> FETCH_HEAD
   adce7b2..4e01ad3 master     -> code-learning/master
Updating adce7b2..4e01ad3
Fast-forward
 test1.py | 3 +++
 1 file changed, 3 insertions(+)

sungt@DESKTOP-4FB6BM4 MINGW64 /d/Code Files/Git Files (master)
$ cat test1.py
name hello i221          master test
hello name i name
hello name i name      main test
this is add by main branch
```

这是一次尝试的修改。拉取远程库。

拉取过来的内容会自动帮你修改更新，提交本地库（`git status`没有提交）

3.4.5 Rebase

在上一节我们看到了，多人在同一个分支上协作时，很容易出现冲突。即使没有冲突，后push的童鞋不得不先pull，在本地合并，然后才能push成功。

每次合并再push后，分支变成了这样：

```
$ git log --graph --pretty=oneline --abbrev-commit
* dlbe385 (HEAD -> master, origin/master) init hello
* e5e69f1 Merge branch 'dev'
| \
| * 57c53ab (origin/dev, dev) fix env conflict
| | \
| | * 7a5e5dd add env
| | * 7bd91f1 add new env
| | /
* | 12a631b merged bug fix 101
| \ \
| * | 4c805e2 fix bug 101
| / /
* | ele9c68 merge with no-ff
| \ \
| | /
| * f52c633 add merge
| /
* cf810e4 conflict fixed
```

`rebase`操作可以把本地未`push`的分叉提交历史整理成直线：

`rebase`的目的是使得我们在查看历史提交的变化时更容易，因为分叉的提交需要三方对比。

3.5 克隆远程库

命令: `git clone`

- 不需要登录账户
- 拉取代码、初始化本地仓库、创建了别名

3.6 团队协作

1) 团队内

将远程仓库的地址复制，打给邀请团队协作的人，对方接受了该仓库的协作请求之后才可以进行拉取和推送代码到远程仓库。

GitHub ——> 设置 ——> 协作者 ——> 邀请

2) 跨团队协作

/ 找到链接 fork 代码 ——> pull request

3.7 SSH免密登录

在C:\Users\sungt文件夹下生成.ssh文件。（右键 bash

`ssh-keygen -t rsa -C sungt2021@gmail.com` 连续三次回车

`ssh-keygen`（生产ssh公和私钥命令） `-t`（用那种加密算法来生产） `rsa`（一种著名的非对称加密协议） `-C`（一种描述） sungt2021@gmail.com

```
sungt@DESKTOP-4FB6BM4 MINGW64 ~
$ ssh-keygen -t rsa -C sungt2021@gmail.com
Generating public/private rsa key pair.

Enter file in which to save the key (/c/Users/sungt/.ssh/id_rsa): Created directory '/c/Users/sungt/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/sungt/.ssh/id_rsa
Your public key has been saved in /c/Users/sungt/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:6pg+s8U9r0aF2u/ICJurcpk7zZa/qWdGT5tJ/CHmWbs sungt2021@gmail.com
The key's randomart image is:
+---[RSA 3072]-----+
|
|   .
|  . .
| +S.
| .ooO o
| =.o+BoX o
| . = BO++O++
| oo*X@*o+oE.
+---[SHA256]-----+
```

```

sungt@DESKTOP-4FB6BM4 MINGW64 ~
$ cd .ssh

sungt@DESKTOP-4FB6BM4 MINGW64 ~/.ssh
$ ll
total 5
-rw-r--r-- 1 sungt 197121 2610 Aug 24 22:13 id_rsa
-rw-r--r-- 1 sungt 197121 573 Aug 24 22:13 id_rsa.pub

sungt@DESKTOP-4FB6BM4 MINGW64 ~/.ssh
$ cat id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDUP5gzMVZKuRDMjdnqi6j4hfmDheyFnLgpTH+YXzDA
MKqYl+91k+YNPA14SvzGApHLr1PU+ris/Aqo4JTKJzwpVMw0iIOBA50TLGt1FB80WUe8VKrdFqic+zn3
88MBcf8n5LdkWVqtoW0Z7reWpkJK5SypZgu1u/VVYVUJSzlt+Bvuh39LsM02CmkVE4kD9wOXnaioZnWd
hsXouAt3PGI7ga2UQJF5VD6Pm4oYv/hGadwHNo56AtGwWS8dwdiE4m56S0LWKvxy71j1ZEzvKZaaHmbf
92rF2137kpSx5ys/KpEugKtVfGfSJP1xjvQVJ9N0a6UtVySaa1+/vYexvkYa70VxYXe5Pp1vSihSLbu4
FoxtkZcFnucpLTnr1V4X1xU7fhPzhyFd6R43Vs2SzNb1zybXuE4SK3GB370SrwAtCmb7y+iY0yzsuvF2
IOY+4PHLaFodji9j13/hEpIoS+02RbaJU09aUQCSTTLBOMVF907EBRXZ3bvw/ESP175yFJk= sungt20
21@gmail.com

```

git pull [git@github.com:Future-001/Code-Learning.git](https://github.com/Future-001/Code-Learning.git) master

4 Git&IDE

4.1 环境配置

4.1.1 配置Git忽略文件

.gitignore文件 2.3 (4)

- 1) 创建忽略规则文件 **xxx.ignore(前缀名随意)**
- 2) 配置 .gitconfig 文件，在其中引用配置忽略文件。

```

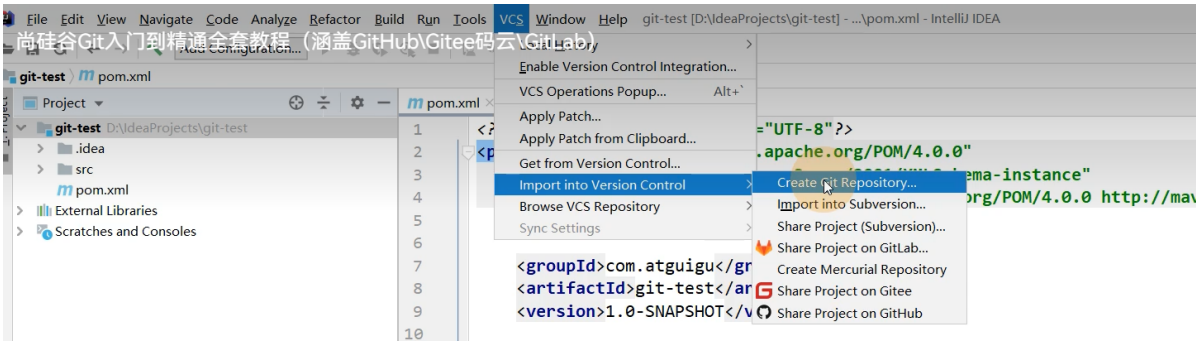
C: > Users > sungt > .gitconfig
1  [user]
2      name = 少主
3      email = sungt2021@gamil.com
4  [core]
5      excludesfile= C:/Users/sungt/git.gitignore

```

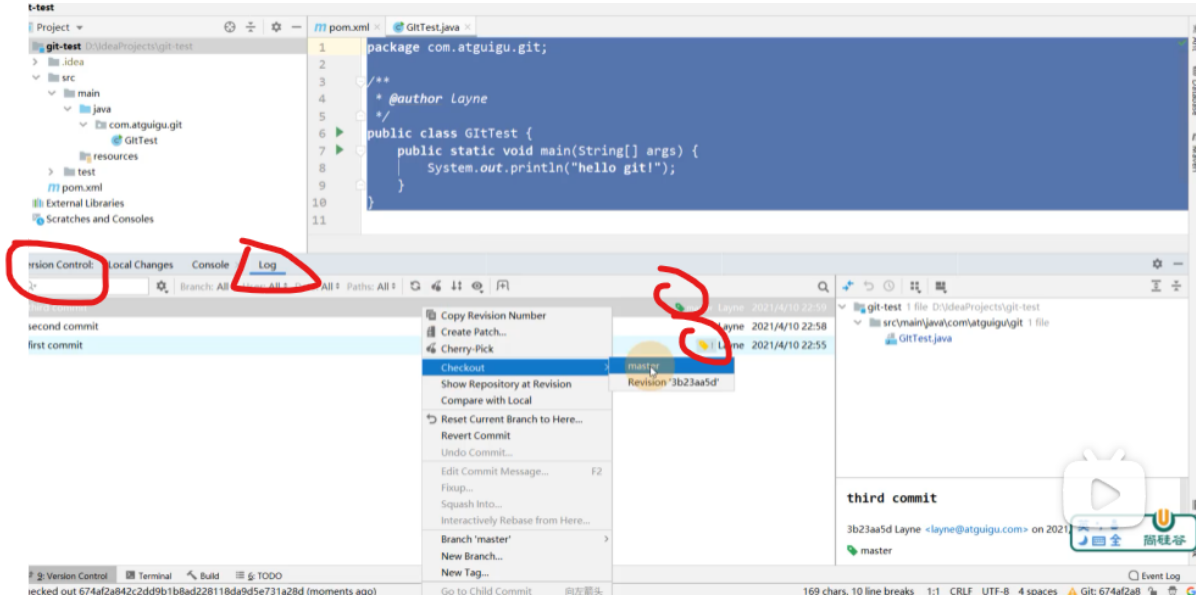


注意斜杠。在IDE里版本控制选择Git的安装环境，选择 git.exe 文件。

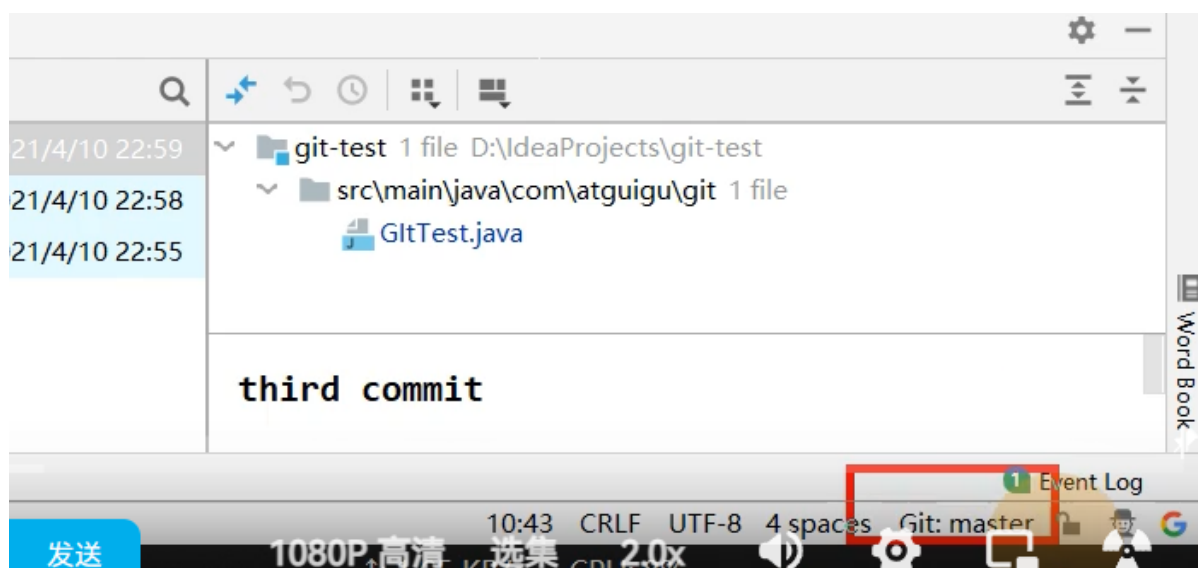
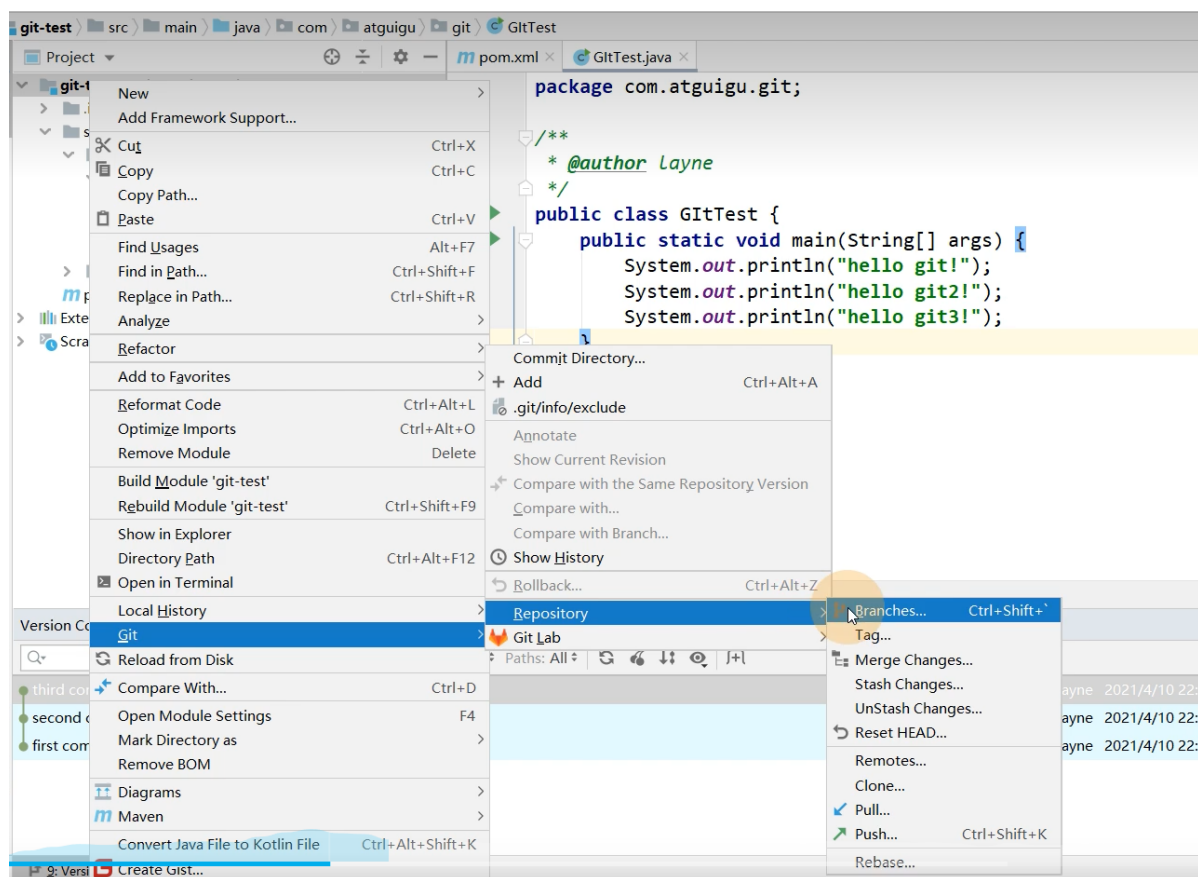
4.2 初始化本地库



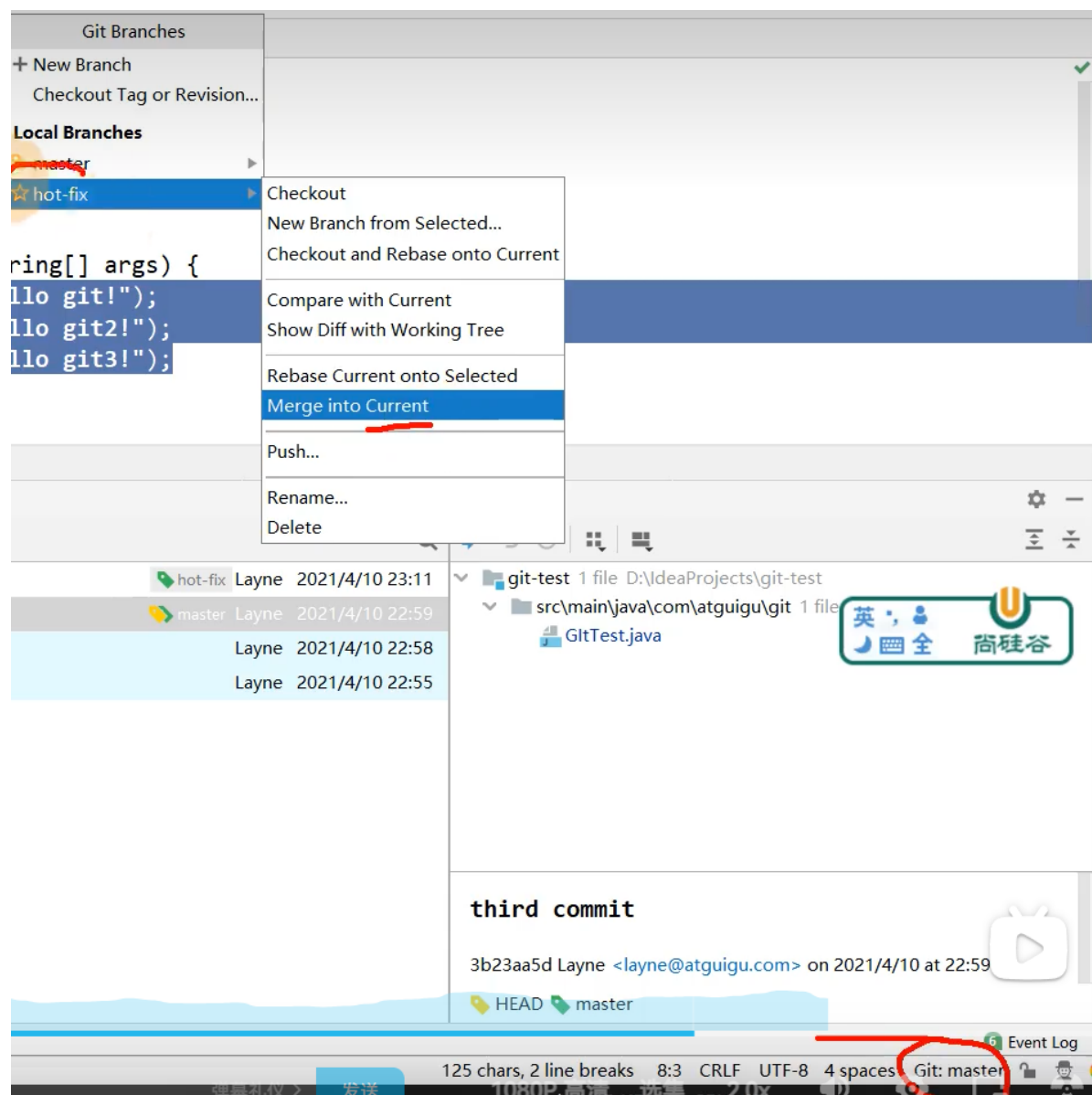
4.3 IDE版本切换



4.4 IDE创建分支



4.5 IDE合并分支



冲突合并：

和上述流程类似，合并时候选择手动合并即可。

4.6 IDE集成GitHub

版本控制里面有GitHub或者plugins也有插件

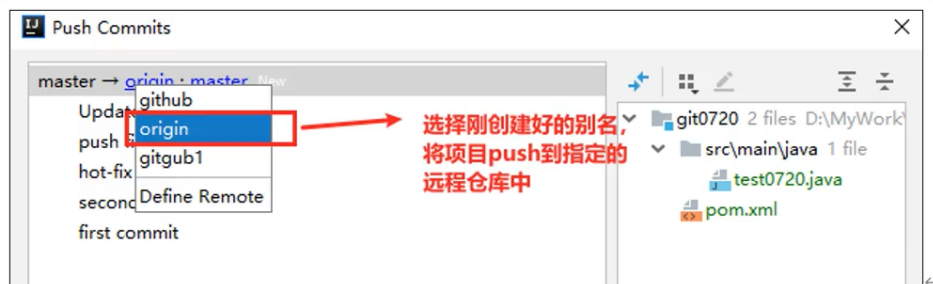
token口令登录。

4.7 IDE分享Guihub

VCS下有分享

4.8 IDE推送&拉取&克隆GitHub项目

利用SSH更快。



注意: **push** 是将本地库代码推送到远程库, 如果本地库代码跟远程库代码版本不一致, **push** 的操作是会被拒绝的。也就是说, 要想 **push** 成功, 一定要保证本地库的版本要比远程库的版本高! 因此一个成熟的程序员在动手改本地代码之前, 一定会先检查下远程库跟本地代码的区别! 如果本地的代码版本已经落后, 切记要先 **pull** 拉取一下远程库的代码, 将本地代码更新到最新以后, 然后再修改, 提交, 推送! ↵



注意: **pull** 是拉取远端仓库代码到本地, 如果远程库代码和本地库代码不一致, 会自动合并, 如果自动合并失败, 还会涉及到手动解决冲突的问题。 ↵

5 Gitee码云

6 Gitlab